

BRIDGE APLICADO EN VEHÍCULOS

Patrones de Diseño: *Bridge*

Desacoplamiento de la abstracción del vehículo
y la implementación de su motor

Autor: **Angie Nathaly Cisneros Guerrero**

20242020084

Fecha: Septiembre 2026

Índice

1. Diagrama UML	2
2. Explicación de las Clases	3
2.1. Jerarquía de Implementación (Motor)	3
2.1.1. Clase Abstracta Motor	3
2.1.2. Implementaciones Concretas	3
2.2. Jerarquía de Abstracción (Vehículo)	5
2.2.1. Clase Abstracta Vehiculo	5
2.2.2. Abstracciones Refinadas	5
3. Código de Cliente	7
3.1. Clase Main	7
4. Pruebas y Ejecución	8
4.1. Prueba de Ejecución en Consola	8
5. Repositorio	9
5.1. Enlace GitHub	9
5.2. Estructura de Paquetes	9

Diagrama UML

Descripción General

El proyecto implementa el patrón de diseño **Bridge**, el cual es un patrón estructural que divide la lógica de negocio en dos jerarquías separadas: la abstracción (los vehículos) y la implementación (los motores). Esto permite que ambas puedan evolucionar de forma independiente.

Diagrama de Clases

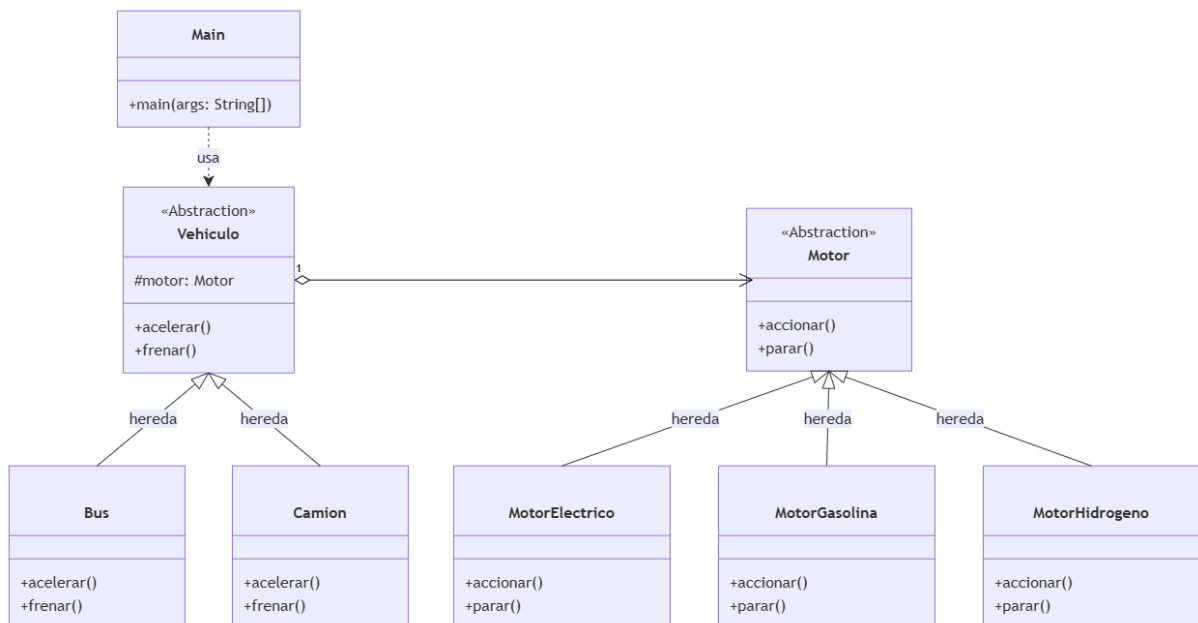


Figura 1: Diagrama de Clases del Sistema (Patrón Bridge).

Explicación de las Clases

Jerarquía de Implementación (Motor)

2.1.1. Clase Abstracta Motor

Descripción

Define la interfaz común para todas las implementaciones de motores. Actúa como el puente que la abstracción utilizará para delegar las tareas de bajo nivel.

Código Fuente

```
1 package vehiculos.bridge;
2
3 public abstract class Motor {
4
5     public abstract void accionar();
6     public abstract void parar();
7
8 }
```

2.1.2. Implementaciones Concretas

Descripción

Clases que heredan de Motor y proveen la lógica específica de encendido y apagado dependiendo del tipo de motor.

Código Fuente: MotorElectrico.java

```
1 package vehiculos.bridge;
2
3 public class MotorElectrico extends Motor{
4
5     @Override
6     public void accionar() {
7         System.out.println("Motor_Electrico: _acelerando_(
8             consumiendoennergia)_");
9     }
10
11     @Override
12     public void parar() {
13         System.out.println("Motor_Electrico: _Cortando_energia");
14     }
15 }
```

Código Fuente: MotorGasolina.java

```
1 package vehiculos.bridge;
2
3 public class MotorGasolina extends Motor{
4
```

```
5      @Override
6      public void accionar() {
7          System.out.println("Motor_a_Gasolina: quemando
8              combustible.");
9      }
10
11      @Override
12      public void parar() {
13          System.out.println("Motor_a_Gasolina: Deteniendo
14              combustion.");
15      }
16  }
```

Código Fuente: MotorHidrogeno.java

```
1  package vehiculos.bridge;
2
3  public class MotorHidrogeno extends Motor{
4
5      @Override
6      public void accionar() {
7          System.out.println("Motor_a_Hidrogeno: Generando energia
8              limpia.(hidrogeno");
9      }
10
11      @Override
12      public void parar() {
13          System.out.println("Motor_a_Hidrogeno: Deteniendo flujo
14              hidrogeno");
15      }
16  }
```

Jerarquía de Abstracción (Vehículo)

Descripción

La clase `Vehiculo` representa la lógica de control de alto nivel. Contiene una referencia (`motor`) a la interfaz de implementación y delega en ella el trabajo. Las clases hijas refinan esta abstracción.

2.2.1. Clase Abstracta Vehiculo

Código Fuente

```
1 package vehiculos.bridge;
2
3 public abstract class Vehiculo {
4
5     protected Motor motor;
6
7     public Vehiculo(Motor motor) {
8         this.motor = motor;
9     }
10
11     public abstract void acelerar();
12     public abstract void frenar();
13
14 }
```

2.2.2. Abstracciones Refinadas

Descripción

Las clases `Bus` y `Camion` extienden la abstracción, agregando sus comportamientos particulares antes de usar el motor asignado.

Código Fuente: Bus.java

```
1 package vehiculos.bridge;
2
3 public class Bus extends Vehiculo{
4
5     public Bus(Motor motor) {
6         super(motor);
7     }
8
9     @Override
10    public void acelerar() {
11        System.out.print("Bus_␣acelerando_␣->␣");
12        this.motor.accionar();
13    }
14
15    @Override
16    public void frenar() {
17        System.out.print("Bus_␣frenando_␣->␣");
```

```
18         this.motor.parar();
19     }
20
21 }
```

Código Fuente: Camion.java

```
1 package vehiculos.bridge;
2
3 public class Camion extends Vehiculo{
4
5     public Camion(Motor motor) {
6         super(motor);
7     }
8
9     @Override
10    public void acelerar() {
11        System.out.print("Camion de carga acelerando->");
12        this.motor.accionar();
13    }
14
15    @Override
16    public void frenar() {
17        System.out.print("Camion de carga frenando->");
18        this.motor.parar();
19    }
20
21 }
```

Código de Cliente

Clase Main

Descripción

La clase **Main** actúa como el **Cliente**. Es responsable de instanciar la implementación concreta (el tipo de motor) y vincularla a la abstracción refinada (el vehículo).

Código Fuente

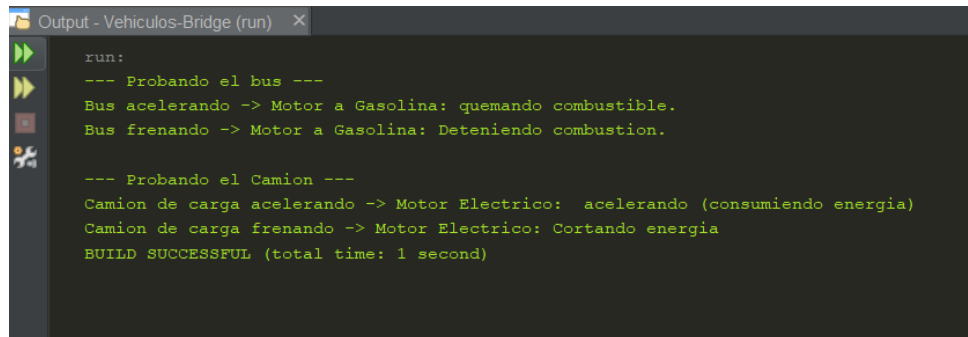
```
1 package vehiculos.bridge;
2
3 public class Main {
4
5     public static void main(String[] args) {
6
7         Motor motorGasolina = new MotorGasolina();
8         Motor motorElectrico = new MotorElectrico();
9
10        Vehiculo bus = new Bus(motorGasolina);
11        Vehiculo camion = new Camion(motorElectrico);
12
13        System.out.println("---_Probando_el_bus_---");
14        bus.acelerar();
15        bus.frenar();
16
17        System.out.println("\n---_Probando_el_Camion_---");
18        camion.acelerar();
19        camion.frenar();
20
21    }
22
23 }
```

Flujo de Ejecución

1. Se instancian componentes concretos de implementación (**MotorGasolina** y **MotorElectrico**).
2. Se crean las instancias de abstracción (**Bus** y **Camion**), inyectándoles el motor a través del constructor.
3. Se invocan los métodos **acelerar()** y **frenar()**, demostrando cómo cada vehículo usa internamente el motor acoplado sin depender de clases fijas.

Pruebas y Ejecución

Prueba de Ejecución en Consola



```
run:
--- Probando el bus ---
Bus acelerando -> Motor a Gasolina: quemando combustible.
Bus frenando -> Motor a Gasolina: Deteniendo combustion.

--- Probando el Camion ---
Camion de carga acelerando -> Motor Electrico: acelerando (consumiendo energia)
Camion de carga frenando -> Motor Electrico: Cortando energia
BUILD SUCCESSFUL (total time: 1 second)
```

Figura 2: Salida de la ejecución mostrando la vinculación dinámica del Bridge.

Repositorio

Enlace GitHub

- <https://github.com/Thalyhx/Modelos-de-Programacion/tree/main/Patrones%20Estructura/Vehiculos-Bridge>

Estructura de Paquetes

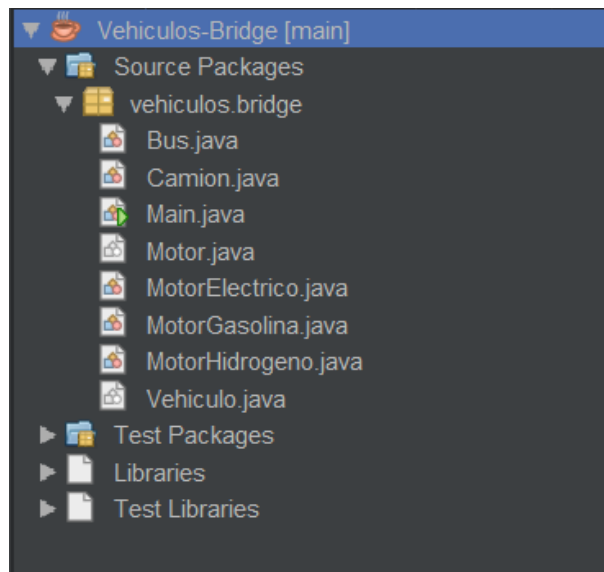


Figura 3: Árbol del proyecto en el IDE.